

13. Alberi di ricerca



Lezione 26 - @November 21, 2025

Alberi di ricerca

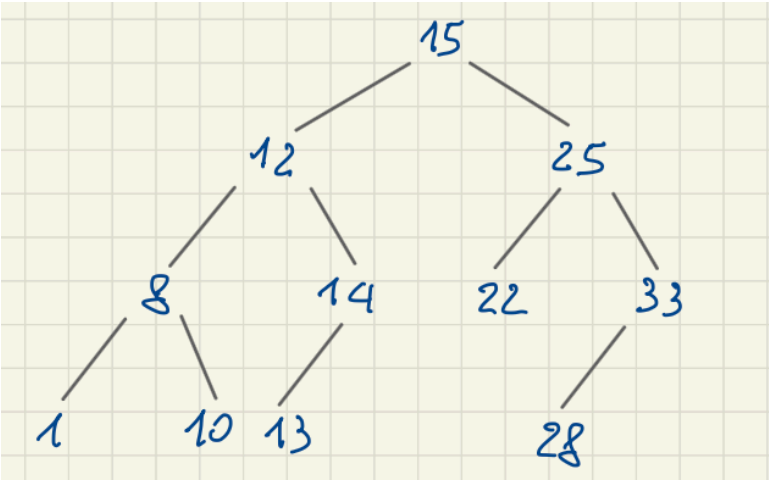
Un *albero binario di ricerca* è un albero binario in cui *per ogni nodo n*:

- il valore di ogni chiave è contenuta nel **sottoalbero sinistro** di *n* è **minore** della chiave contenuta in *n*.
- il valore di ogni chiave è contenuta nel **sottoalbero destro** di *n* è **maggiore o uguale** della chiave contenuta in *n*.

alberi binari - **no heap**

con la visita in profondità in ordine simmetrico (sx - radice - dx) restituisce una lista ordinata.

Visite negli alberi



- visita in ampiezza
15 12 25 8 14 22 33 1 10 13 28
- visita in profondità in ordine simmetrico

sx - radice - dx

1 8 10 12 13 14 15 22 25 28 33

Ricerca massimo e minimo (chiavi intere)

```
FUNZIONE massimo(AlberoRicerca r) -> Nodo
  IF r = null THEN RETURN null
  ELSE
    n <- r
    WHILE n.dx != null DO
      n <- n.dx
    RETURN n
```

```
FUNZIONE minimo(AlberoRicerca r) -> Nodo
  IF r = null THEN RETURN null
  ELSE
    n <- r
    WHILE n.sx != null DO
      n <- n.sx
    RETURN n
```

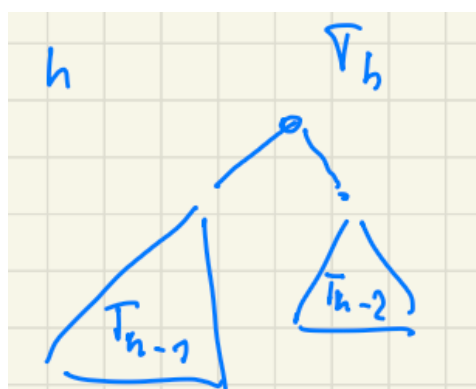
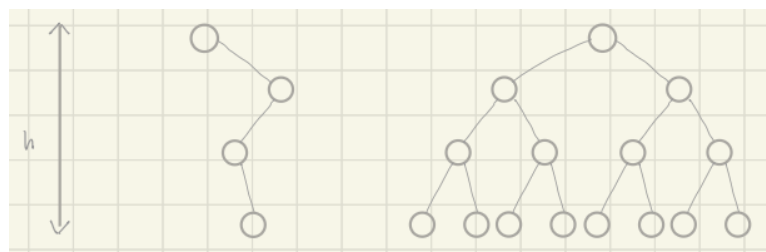
Il tempo dipende dall'altezza dell'albero

Ricerca ricorsiva

```
FUNZIONE trova(AlberoRicerca r, TipoChiave k) -> Nodo
  IF r = null THEN
    RETURN null
  ELSE IF k < r.chiave THEN
    RETURN trova(r.sx, k)
  ELSE IF k > r.chiave THEN
    RETURN trova(r.dx, k)
  ELSE RETURN r
```

ricorsioni in coda → si possono eliminare

il numero di passi dipende dall'altezza.



Lezione 27 - @November 24, 2025

Ricerca Iterativa

eliminazione della ricorsione nella ricerca ricorsiva per alberi binari di ricerca

```
FUNZIONE trova(AlberoRicerca r, tipoChiave k) -> Nodo
  n <- r
  WHILE n != null AND chiave n.chiave != k DO
    IF k < n.chiave THEN n <- n.sx
    ELSE n <- n.dx
  RETURN n
```

Inserimento

```
FUNZIONE inserisci(AlberoRicerca r, elemento d) -> AlberoRicerca
  k <- d.chiave
  t <- riferimento a un nuovo nodo
  t.chiave <- k
  t.altricampi <- d.altricampi
  t.sx <- null
  t.dx <- null

  padre <- null
  n <- r
  WHILE n != null AND n.chiave != k DO
    padre <- n
```

```
IF k < n.chiave THEN n <- n.sx
ELSE n <- n.dx
IF padre = null THEN
  r <- b
ELSE IF k < padre.chiave THEN
  padre.sx <- t
ELSE padre.dx <- t
RETURN r
```

Cancellazione

1. il nodo è una foglia → facile
caso particolare: cancello radice
2. il nodo da cancellare ha un unico figlio: modifica il puntatore nel nodo padre
caso particolare: cancello radice
3. normalmente: sostituisci con il contenuto del nodo chiave più piccolo del sottoalbero destro del nodo da cancellare.
Il nodo utilizzato ha al massimo 1 figlio → rimuovilo (caso 1 o 2)

Alberi binari: n° nodi vs altezza

- n numero di nodi
- h altezza

Che relazioni ci sono tra numero di nodi e altezza un albero binario?

$$h + 1 \qquad 2^{h+1} - 1$$

albero degenerare

albero completo

$$h + 1 \leq n \leq 2^{h+1} - 1$$

$$\begin{aligned} h &\leq n - 1 \\ h &< n \end{aligned}$$

$$\begin{aligned} n &< 2^{h+1} \\ \log_2 n &< h + 1 \\ \lfloor \log_2 n \rfloor &\leq h \end{aligned}$$

$$\lfloor \log_2 n \rfloor \leq h < n$$

Alberi perfettamente bilanciati



Un albero binario è detto perfettamente bilanciato quando per ogni nodo la differenza in valore assoluto tra i numeri di nodi presente nei suoi sottoalberi sinistro e destro è al massimo 1

Altezza sempre logaritmica

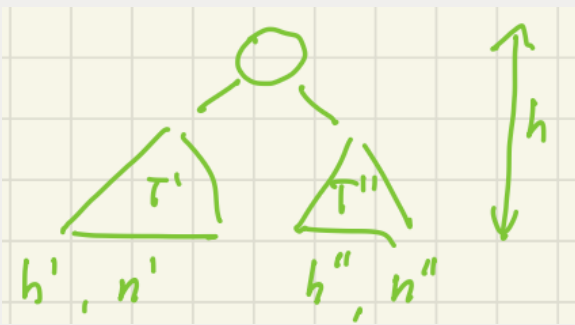
A. numero minimo di nodi: $n \geq 2^h$



dimostrazione - induzione su h

base $h = 0$ 1 nodo $1 \geq 2^0$ - OK

induzione $h - 1 \rightarrow h$



- almeno uno tra T' e T'' deve avere altezza $h - 1$
- supponiamo T' , per ipotesi di induzione $n' \geq 2^{h'} = 2^{h-1}$
poiché T è perfettamente bilanciato $n'' \geq n' - 1$

$$n = 1 + n' + n'' \geq 1 + 2^{h-1} + 2^{h-1} - 1 = 2 \cdot 2^{h-1} = 2^h \quad \square$$

B. $n < 2^{h+1}$ (numero massimo di nodi: albero completo)

$$2^h \leq n < 2^{h+1} \quad h \leq \log_2 n < h + 1 \Rightarrow h = \lfloor \log_2 n \rfloor$$

problema con gli alberi perfettamente bilanciati:

l'inserimento può chiedere la ristrutturazione dell'intero albero $\Rightarrow$ **troppo costoso!**

Alberi bilanciati in altezza



Un albero

binario è detto bilanciato (in altezza) o AVL¹ quando per ogni nodo la differenza in valore assoluto tra le altezze dei suoi sottoalberi sinistro e destro è al massimo 1.

¹Adelson-Velsky and Landis, 1926

Nota perfettamente bilanciato $\Rightarrow$ bilanciato.

Alberi bilanciati in altezza: n° nodi vs altezza

numero massimo di nodi $\rightarrow$ albero completo di altezza h : $n = 2^{h+1} - 1$

numero minimo di nodi:

- $h = 0$ $n_0 = 1$
- $h = 1$ $n_1 = 2$
- $h = 2$ $n_2 = 4$
- $h = 3$ $n_3 = 7$

Albero di fibonacci di altezza h $\rightarrow$ albero AVL di altezza minima

$$n_h := \# \text{ nodi di } T_h = \begin{cases} 1 & h = 0 \\ 2 & h = 1 \\ 1 + n_{h-1} + n_{h-2} & \text{altrimenti} \end{cases}$$



dimostriamo che $n_h = F_{h+3} - 1$, con $F_1 = F_2 = 1, F_k = F_{k-1} + F_{k-2}$

induzione

base:

$$\left| \begin{array}{lll} h = 1 & n_0 = 1 & F_3 = 2 \\ h = 2 & n_1 = 2 & F_4 = 3 \end{array} \right.$$

$< h \rightarrow h$

$$\left| \quad n_h = 1 + n_{h-1} + n_{h-2} \right.$$

per ipotesi di induzione:

$$\left| \quad n_h = 1 + F_{h+2} - 1 + F_{h+1} - 1 = F_{h+3} - 1 \right.$$

□

$$\begin{aligned} n_h &\approx \frac{\phi^{h+3}}{\sqrt{5}} & \sqrt{5}n_h &\approx \phi^{h+3} \\ \log_{\phi}(\sqrt{5}n_h) &\approx h+3 & h &\approx \log_{\phi} \sqrt{5} - 3 + \log_{\phi}(n_h) \\ h &= \Theta(\log n) \Rightarrow \text{Ricerca } \Theta(\log n) \end{aligned}$$



Lezione 28 - @November 26, 2025

alberi perfettamente bilanciati $h = \lfloor \log_2 n \rfloor$

mantenere la proprietà di bilanciamento è molto costoso!

alberi bilanciati in altezza [AVL]: quando per ogni nodo la differenza in valore assoluto tra le altezze dei suoi sottoalberi sinistro e destro è al massimo 1

alberi di fibonacci di altezza h

il numero di nodi in un albero di fibonacci con altezza h è

$$\begin{aligned} n_h &\approx \frac{\phi^{h+3}}{\sqrt{5}} \\ \Rightarrow h &= O(\log n) \\ \text{Si può dimostrare che } h &= 1.44 \log_2 n \end{aligned}$$

Alberi AVL: altezza $\Theta(\log n)$

$\Rightarrow$ costo ricerca $\Theta(\log n)$

Inserimento in alberi AVL

Si deve gestire il **fattore di bilanciamento**:

In un albero AVL, per ogni nodo, la differenza tra l'altezza del sottoalbero sx e quella del sottoalbero dx deve essere **-1, 0 o +1**.

$$BF(nodo) = Altezza(Sinistra) - Altezza(Destra)$$

Se $|BF| > 1$, l'albero è sbilanciato e richiede una "riparazione" tramite rotazioni.

algoritmo di inserimento

L'inserimento avviene in due fasi principali: una fase di discesa (standard BST) e una fase di risalita (controllo e bilanciamento).

- 1. **Inserimento Standard (BST):** Si inserisce il nuovo elemento come foglia, seguendo la logica minore/maggiore (sinistra/destra), esattamente come in un BST normale.
- 2. **Aggiornamento Altezze (Backtracking):** Risalendo ricorsivamente dalla nuova foglia verso la radice, si aggiorna l'altezza di ogni nodo genitore visitato.
- 3. **Controllo del Bilanciamento:** Per ogni nodo durante la risalita, si calcola il Fattore di Bilanciamento (BF).
- 4. **Rotazioni (Se necessario):** Se un nodo ha un BF di **+2** o **-2**, si applica una rotazione per riequilibrare quel sottoalbero.

i 4 casi di sbilanciamento e le rotazioni

Quando un nodo z diventa sbilanciato dopo un inserimento, ci sono 4 scenari possibili. Supponiamo che y sia il figlio di z e x sia il nipote (il nodo appena inserito o quello nel percorso di inserimento).

Caso 1: Sinistra-Sinistra (LL - Left-Left)

Lo sbilanciamento è causato dall'inserimento nel sottoalbero **sinistro** del figlio **sinistro**.

- **Situazione:** Il nodo è "pesante a sinistra" ($BF = +2$) e anche suo figlio è pesante a sinistra ($BF = +1$ o 0).
- **Soluzione:** Rotazione Semplice a Destra.

Caso 2: Destra-Destra (RR - Right-Right)

Lo sbilanciamento è causato dall'inserimento nel sottoalbero **destro** del figlio **destro**.

- **Situazione:** Il nodo è "pesante a destra" ($BF = -2$) e anche suo figlio è pesante a destra ($BF = -1$ o 0).
- **Soluzione:** Rotazione Semplice a Sinistra.

Caso 3: Sinistra-Destra (LR - Left-Right)

Lo sbilanciamento è causato dall'inserimento nel sottoalbero **destro** del figlio **sinistro**.

- **Situazione:** Il nodo padre è pesante a sinistra ($BF = +2$), ma il figlio è pesante a destra ($BF = -1$). L'albero ha una forma a "gomito" o zig-zag.
- **Soluzione: Doppia Rotazione (LR).**
 - 1. Rotazione a **Sinistra** sul figlio sinistro.
 - 2. Rotazione a **Destra** sul nodo sbilanciato originale.

Caso 4: Destra-Sinistra (RL - Right-Left)

Lo sbilanciamento è causato dall'inserimento nel sottoalbero **sinistro** del figlio **destro**.

- **Situazione:** Il nodo padre è pesante a destra ($BF = -2$), ma il figlio è pesante a sinistra ($BF = +1$).
- **Soluzione: Doppia Rotazione (RL).**
 - 1. Rotazione a **Destra** sul figlio destro.
 - 2. Rotazione a **Sinistra** sul nodo sbilanciato originale.

Complessità Temporale

Grazie a questo meccanismo, l'inserimento in un AVL è molto efficiente:

- **Ricerca della posizione:** $O(\log n)$
- **Ribilanciamento (Rotazioni):** $O(1)$ (dopo l'inserimento, al massimo sono necessarie poche rotazioni locali, anche se il ricalcolo dell'altezza risale fino alla radice).
- **Totale:**

$O(\log n)$

Riassumendo: Alberi AVL

Altezza $\Theta(\log n)$

Tempo	
Ricerca	$\Theta(\log n)$
Inserimento	$\Theta(\log n)$

Tempo	
Cancellazione	$\Theta(\log n)$



Strutture più efficienti per i dizionari (fino ad ora)

Alberi 2-3



Un albero 2-3 è un albero in cui:

- ogni nodi interno ha 2 o 3 figli,
- tutte le foglie si trovano allo stesso livello

numero di nodi/foglie vs altezza

un albero con il minimo numero di nodi/foglie è un albero un albero con il massimo numero di nodi/foglie binario completo di altezza h

$$\begin{aligned} \# \text{nodi} &: 2^{h+1} - 1 \\ \# \text{foglie} &: 2^h \end{aligned}$$

$$\begin{aligned} \# \text{nodi} &: \frac{3^{h+1} - 1}{2} \\ \# \text{foglie} &: 3^h \end{aligned}$$

Se ci sono n foglie: $2^h \leq n \leq 3^h$

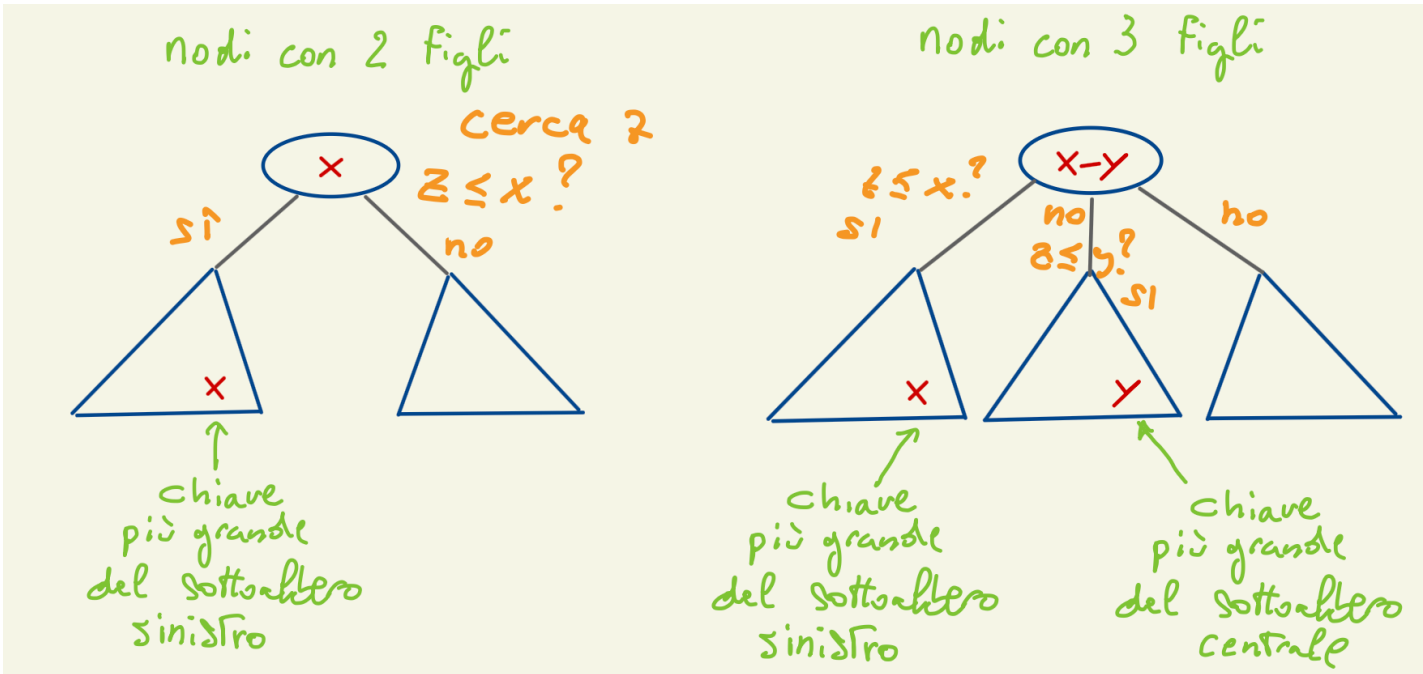
$$\log_3 n \leq h \leq \log_2 n$$

Pertanto l'altezza è logaritmica rispetto al numero di foglie

$$h = \Theta(\log n)$$

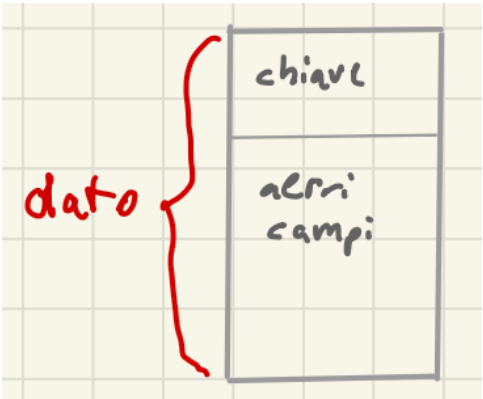
Alberi 2-3 di ricerca

- Dati memorizzati esclusivamente nelle foglie, in ordine non decrescente da sinistra verso destra.
- I nodi interni contengono alcune chiave, utilizzate come informazioni di *instradamento*.

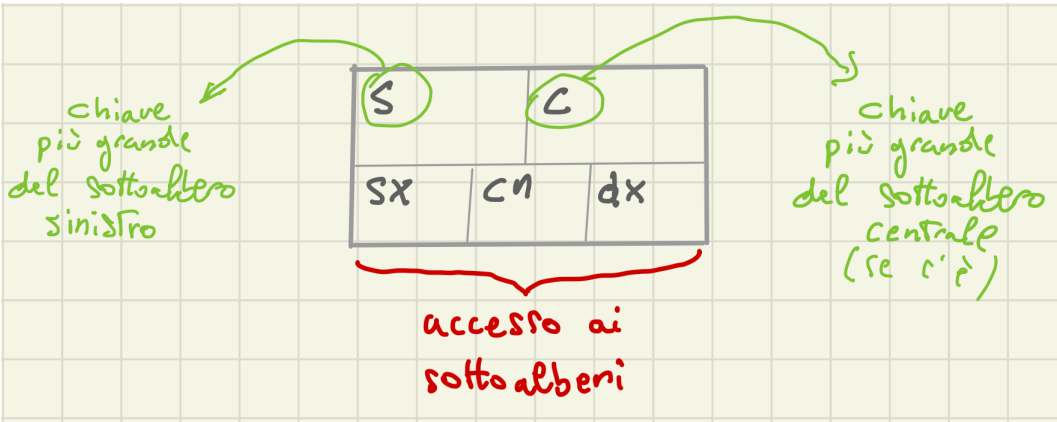


Rappresentazione

- foglie



- nodi interni



per inserimenti/cancellazioni è utile memorizzare in ogni nodo anche un puntatore al padre.

Costo delle operazioni per gli alberi 2-3

Nome operazione	Tempo
Ricerca	$\Theta(\log n)$
Inserimento	$\Theta(\log n)$
Cancellazione	$\Theta(\log n)$



Lezione 29 - @November 28, 2025

B-Alberi



Alberi di Bayer

Dati su memoria secondaria

DBMS (data base management system) usano indici per garantire accesso più veloce ai dati. Possibilità di avere più indici, basati su campi differenti.

Indici possono essere molto grandi (migliaia o milioni di chiavi). A causa delle loro dimensioni non possono essere caricati in memoria centrale.

Dizionari possono essere realizzate strutture ad albero per la loro implementazione (per rappresentare indici) in memoria secondaria. È necessario tenere conto di vincoli tecnologici.

Vincoli tecnologici

- accesso ai dati più lento che in memoria centrale.

Nei dischi i tempi di latenza dovuti a parti meccaniche sono circa 100.000 volte più grandi rispetto alla memoria centrale.

accesso al singolo dato più lento

- accesso a blocchi (o pagine)

Dimensioni tipiche $2^9 = 512$ byte, $\dots 2^{12} = 4096$ byte.

in un unico accesso si ottengono molto dati

B-Alberi definizione

Un albero di ordine t (grado minimo t) è un albero T tale che:

- ogni *nodo interno* ha al massimo $2t$ figli,
- ogni *nodo interno diverso* dalla radice ha almeno t figli,
- la radice ha almeno 2 figli,
- tutte le foglie si trovano allo stesso livello **oppure** T è l'albero vuoto.
- Ogni foglia contiene k chiavi ordinate $a_1 < a_2 < \dots < a_k$ dove $t - 1 \leq k \leq 2t - 1$ ($1 \leq k \leq 2t - 1$ se la foglia è la radice)
- Ogni nodo interno con $k + 1$ figli e sottoalberi $T_0, T_1, \dots, T_k$ contiene k chiavi ordinate $a_1 < a_2 < \dots < a_k$ tali che, per ogni chiave c_i di $T_i, i = 0, \dots, k$ risulta:

$$c_0 < a_1 < c_1 < a_2 < \dots < c_{k-1} < a_k < c_k$$

Ricerca

- Nodo: array ordinato con k chiavi, $k < 2t \rightarrow$ Ricerca Binaria in $\Theta(\log t)$ passi

Ricerca Binaria:

- chiave presente $\rightarrow$ FINITO
- chiave assente:
 - foglia $\rightarrow$ FINITO
 - nodo interno:
 - si prosegue in sottoalbero.
- Altezza dell'albero $h \rightarrow$ si riparte la massimo h volte.

$$\# \text{totale di passi} = \Theta(h \log t)$$

Quanto può essere al massimo h con n chiavi?

h	nodi	numero di chiavi n
0	1	1
1	+2	+2($t - 1$)
2	+2 t	+2 t ($t - 1$)
3	2 t^2	+2 t^2 ($t - 1$)
$\vdots$	$\vdots$	$\vdots$
h	+2 t^{h-1}	+2 t^{h-1} ($t - 1$)

$$\begin{aligned} &\Rightarrow 1 + 2 \sum_{i=0}^{h-1} t^i (t - 1) \\ &= 1 + 2(t - 1) \sum_{i=0}^{h-1} t^i = 1 + 2(t - 1) \frac{t^h - 1}{t - 1} = 1 + 2t^h - 2 = 2t^h - 1 \\ &\Rightarrow n \geq 2^t - 1 \\ &n \geq 2t^h - 1 \qquad t^h \leq \frac{n + 1}{2} \qquad h \leq \log_t \frac{n + 1}{2} \end{aligned}$$

Se ci sono n chiavi l'altezza h è al massimo $\log_2 \frac{n + 1}{2}$

passi totali per la ricerca $\Theta(h \log t)$ con $h \leq \log_t \frac{n+1}{2}$

$$h \log_b t \leq \log_t \frac{n+1}{2} \cdot \log_b t = \log_b \frac{n+1}{2}$$

passi di ricerca $\Theta(\log n) \leftarrow$ indipendente da t !

Accessi a memoria secondaria in B-alberi

- scegliendo t in modo che ogni nodo sia contenuto **internamente** in un blocco
- numero di accesso $\approx$ altezza blocco $= \log_t n$

se si usassero alberi AVL o 2-3? numero di accessi $\approx \log_2 n$

per $10^6 = 1$ milione di chiavi $\approx 2^{20}$

→ AVL $\log_2 10^6 \approx \log_2 2^{20} = 20$ accessi a memoria di massa

→ B-Albero $t = 100$,

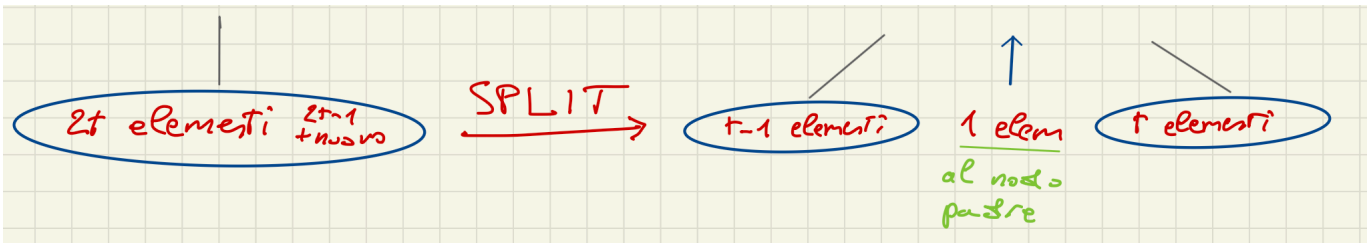
→ B-Albero $t = 100$, #accessi $\approx \log_{100} 10^6 = 3$

Inserimento



ricerca della foglia in cui l'elemento va inserito

1. c'è spazio nella foglia → inserisci l'elemento al posto giusto
2. non c'è spazio nella foglia (ci sono già $2t - 1$ elementi)



Riepilogo costi

operazione	passi di calcolo	accessi a memoria di massa
ricerca	$\Theta(\log n)$	$\approx \log_t n$
Inserimento/cancellazione	$\Theta(t \cdot \log_t n)$	$\approx C \cdot \log_t n$, C costante piccola (dipende dall'implementazione), solitamente $C \approx 4$.

Costi delle operazioni sui dizionari

numero di confronti in funzione del numero di chiavi

	Array ordinati	Alberi binari di ricerca	Alberi AVL Alberi 2-3
Ricerca	$\Theta(\log n)$	$\Theta(n)$	$\Theta(\log n)$
Inserimento	$\Theta(n)$	$\Theta(n)$	$\Theta(\log n)$
Cancellazione	$\Theta(n)$	$\Theta(n)$	$\Theta(\log n)$
svantaggio Capacità fissa		costo per i puntatori	

